ELEVAI TECHNICAL DEFENSE

Master Viva & Backend Architecture Questionnaire

Complete Guide: System Architecture, AI Pipelines, Backend Workflows, Database & Defense Answers

SECTION 1: BACKEND REQUEST-RESPONSE LIFECYCLE & EXECUTION

Step-by-step trace of incoming requests through the Express middleware pipeline

Q1: Walk me through the exact lifecycle of an HTTP request when it hits your backend server.

When a candidate or admin performs any action in the React frontend, the request travels through this exact step-by-step pipeline:

- CORS & Security Check: The request arrives at Node.js/Express. cors() validates if the origin is authorized (localhost:5173 or production domain).
- Body Parsing: express.json() and express.urlencoded() parse JSON payloads into accessible req.body objects.
- Multer Interception (If File Upload): For resume uploads, multer intercepts multipart/form-data, validates MIME-type (application/pdf) & size (<= 5MB), and loads the file into memory as req.file.buffer.
- Authentication Middleware (authenticateToken): Extracts "Authorization: Bearer <token>", verifies the signature using jwt.verify() with JWT_SECRET, and attaches decoded user to req.user.
- Role Authorization Guard (requireAdmin): On admin routes, checks req.user.role === "admin". Blocks candidates with HTTP 403 Forbidden.
- Route Controller & Service Logic: Executes core business logic, database queries via Supabase client, or external AI API calls to Gemini.
- Response Dispatch: Returns standardized JSON payload { success: true, data: ... } with HTTP 200/201.
- Global Error Middleware: If any unhandled exception occurs, the central error middleware catches it and returns HTTP 500 without crashing.

EXAMINER TIP: Mention the exact order: CORS -> JSON Parser -> Multer -> JWT Auth -> RBAC -> Controller -> Database/AI -> Response.

Q2: How does the backend handle asynchronous operations without blocking the single-threaded Node.js event loop?

Node.js runs on a single-threaded non-blocking Event Loop. All I/O-intensive operations (database queries in Supabase, API calls to Google Gemini, and cloud storage file uploads) are written using async/await and Promises.

- Non-Blocking Offloading: When an async task begins, Node offloads the socket I/O to libuv background threads, freeing the main thread to handle other users.
- Microtask Callback Queue: When Gemini or PostgreSQL finishes, the callback is pushed to the event microtask queue and resumes execution immediately.

SECTION 2: RESUME PROCESSING & ATS SCORER BACKEND WORKFLOW

Detailed mechanics of file upload, memory buffering, text parsing, and AI analysis

Q3: Explain the step-by-step backend workflow when a candidate uploads a resume.

The resume upload pipeline processes the file entirely in-memory and pushes it to both cloud storage and the AI engine:

- 1. Form-Data Upload: Client sends POST /api/resumes/upload with multipart/form-data containing the PDF and Bearer JWT.
- 2. In-Memory Buffering: Multer memoryStorage intercepts the file stream, validates application/pdf MIME-type and <= 5MB limit, and buffers it in req.file.buffer.
- 3. Text Extraction: The backend passes the memory buffer to pdf-parse, extracting raw resume text, sections, and bullet points into a clean string.
- 4. Cloud Storage Upload: The binary buffer is streamed to the Supabase Storage private bucket under "resumes/{userId}_{timestamp}.pdf", generating a secure URL.
- 5. AI ATS Scoring: The extracted text is injected into an ATS prompt and sent to Google Gemini 1.5 Flash.

- 6. JSON Sanitization & Validation: The backend cleans markdown backticks and validates { ats_score (0-100), strengths, improvements, missing_keywords }.
- 7. Database Persistence: An INSERT query saves user_id, file_url, file_name, extracted_text, ats_score, and feedback_json into the resumes table.
- 8. Response: Returns HTTP 201 Created with the calculated ATS score and full scorecard.

Q4: Why do you process the PDF in memory with Multer instead of saving it to disk first?

Processing in memory provides three critical engineering advantages:

- Ephemeral Cloud Disk Compatibility: Cloud platforms (Render, Heroku) have temporary file systems that wipe local files on restart or scaling.
- High Performance & Zero Disk I/O: In-memory buffer operations avoid slow physical hard drive read/write cycles, cutting latency by 3x.
- Security & Privacy: Candidate resumes are never left exposed in local server folders where they could lead to data leaks.

SECTION 3: AI MOCK INTERVIEW ENGINE BACKEND WORKFLOW

Question generation, answer evaluation, prompt resilience, and rubrics

Q5: How is an AI mock interview generated and saved in the backend?

The interview creation lifecycle executes in 5 precise steps:

- 1. Payload Reception: Client sends { domain: "Fullstack", difficulty: "Medium", count: 5 } to POST /api/interview/generate.
- 2. Prompt Construction: Backend builds a strict zero-shot prompt requiring Gemini to return ONLY a valid JSON array of interview questions.
- 3. AI API Call: Calls aiModel.generateContent(prompt) using @google/genai / @google/generative-ai SDK.
- 4. Regex Sanitizer & Parser: Backend removes markdown ticks (```json) and validates array integrity via JSON.parse().
- 5. DB Insertion: Inserts a row in interviews table with status = "in_progress", user_id, domain, and questions JSONB, returning the interviewId.

Q6: What is the exact workflow when the candidate submits their interview answers?

Answer submission evaluates 4 distinct rubrics and records the candidate scorecard:

- 1. Submission: Client sends POST /api/interview/submit with { interviewId, answers } containing speech-to-text transcripts or typed answers.
- 2. Ownership Verification: Backend confirms the interviewId belongs to req.user.id and is currently "in_progress".
- 3. AI Rubric Evaluation: Prompts Gemini 1.5 Flash to grade against 4 categories: Domain Knowledge (0-100), Clarity (0-100), Confidence (0-100), and Communication (0-100).
- 4. Scorecard Validation: Backend ensures all numeric scores are within 0-100 and validates structured feedback arrays.
- 5. Database Update: Executes UPDATE interviews SET status="completed", overall_score, category_scores, feedback WHERE id=:interviewId.
- 6. Response: Returns HTTP 200 OK with the final structured scorecard and personalized improvement roadmap.

SECTION 4: JOB APPLICATION & STATE MACHINE BACKEND WORKFLOW

6-Stage state engine, duplicate guards, audit trails, and automated notifications

Q7: What happens in the backend when a candidate applies for a job?

The application submission workflow enforces duplicate prevention and snapshot immutability:

- 1. Duplicate Check: Backend queries applications table for WHERE job_id = :jobId AND user_id = :userId. If exists, returns HTTP 400 "Already applied".
- 2. Resume Snapshot Binding: Fetches candidate's chosen resume and captures an immutable snapshot of the file URL and ATS score at application time.
- 3. DB Record Creation: Executes INSERT into applications (job_id, user_id, resume_id, status="applied", resume_snapshot).
- 4. Response: Returns HTTP 201 Created with application tracking ID.

Q8: How does the backend execute an application status transition and notify the candidate?

Status updates follow an atomic, governed transition pipeline:

- 1. RBAC Guard: requireAdmin middleware validates caller is an authenticated recruiter.
- 2. State Validation: Verifies target status is valid enum: applied -> reviewed -> shortlisted -> interview_scheduled -> accepted / rejected.
- 3. Database Update: Executes UPDATE applications SET status = :status, notes = :notes WHERE id = :id.
- 4. Audit Logging: Inserts an entry into audit_logs table (admin_id, application_id, old_status, new_status, timestamp) for compliance.
- 5. Asynchronous Notification: Inserts a row into notifications table (user_id=candidateId, title="Status Updated", message) for in-app alert.
- 6. Response: Returns HTTP 200 OK confirming update and notification dispatch.

SECTION 5: AUTHENTICATION & RBAC MIDDLEWARE EXECUTION FLOW

Bcrypt password hashing, stateless JWT verification, and route guards

Q9: Explain how authenticateToken and requireAdmin work at the code level.

Authentication and authorization are separated into two distinct, composable middlewares:

- authenticateToken: Reads req.headers["authorization"], validates "Bearer <token>" format, and calls jwt.verify(token, JWT_SECRET). If valid, attaches { id, email, role } to req.user and calls next(). If expired/invalid, returns HTTP 401.
- requireAdmin: Assumes authenticateToken ran first. Checks if req.user.role === "admin". If true, calls next(). If role is "candidate", halts with HTTP 403 Forbidden.

Q10: How are passwords stored and verified securely during login?

Passwords are protected using industry-standard salted cryptographic hashing:

- Registration: Plaintext passwords are never stored. The backend runs bcrypt.hash(password, 10) with 10 salt rounds to generate an irreversible 60-character hash.
- Login: When a user logs in, the backend retrieves the stored hash by email and executes bcrypt.compare(candidatePassword, storedHash). Upon match, a signed JWT is minted.

SECTION 6: DATABASE SCHEMA & RELATIONAL QUERY EXECUTION

7 PostgreSQL tables, Supabase SDK, foreign keys, and cascade deletes

Q11: Explain your database schema. How many tables are there and how are they related?

ElevAI uses 7 relational tables in PostgreSQL managed via Supabase:

- users: Credentials, email, password hash, role ("candidate" / "admin"), timestamps.
- resumes: Resume metadata, Supabase storage URL, extracted text, ATS score, feedback. (FK -> users.id)
- interviews: Mock interview sessions, domain, difficulty, questions, answers, category scores, feedback. (FK -> users.id)
- jobs: Job postings created by admins (title, company, description, requirements, salary, status). (FK -> users.id)
- applications: Bridges candidates and jobs with 6-stage status, resume snapshot, recruiter notes. (FK -> jobs.id, users.id, resumes.id)
- notifications: In-app alerts for status updates and interview schedules with read/unread flag. (FK -> users.id)
- audit_logs: Immutable log of administrative actions (status changes, job deletion) for accountability. (FK -> users.id)

Q12: Why did you use Supabase Storage instead of saving PDF files directly inside the database as BLOBs?

Database BLOB storage creates severe bottlenecks: it consumes precious database RAM, slows down table scans, and bloats backups. Storing binary files in Supabase S3-compatible Object Storage while keeping only the URL and parsed text in PostgreSQL is the scalable industry standard.

SECTION 7: ERROR HANDLING, LOGGING & FAULT TOLERANCE

Try-catch strategies, global error middlewares, and AI timeout fallbacks

Q13: How does your backend handle uncaught exceptions and external service outages?

ElevAI implements multi-tiered fault tolerance:

- Controller Try-Catch: Every asynchronous route wraps logic in try-catch to prevent unhandled promise rejections.
- Global Express Error Handler: At the bottom of the server file, a 4-argument error middleware (err, req, res, next) intercepts unexpected errors and sends clean JSON responses.
- AI Outage Fallback: If Gemini API encounters rate limits (HTTP 429) or network timeouts, the backend logs the event and returns a structured fallback message without crashing.

SECTION 8: GENERAL & NON-TECHNICAL PROJECT QUESTIONS

Core value proposition, dual workflows, and market differentiation

Q14: What is the core value proposition of ElevAI and what makes it unique?

ElevAI bridges preparation and recruitment into a single closed-loop ecosystem. Candidates practice with real-time AI mock interviews and ATS resume analysis, while recruiters review comprehensive candidate dossiers containing verified AI interview scores alongside resumes before scheduling live interviews.

Q15: What are the two primary user workflows in the system?

The application caters to two distinct user journeys:

- Candidate Journey: Register -> Take AI Voice Mock Interview -> Upload Resume for ATS Scoring -> Browse Job Marketplace -> Submit Application -> Receive Real-Time Notifications.
- Admin/Recruiter Journey: Login as Admin -> Post Jobs -> Open Candidate Review Dossier -> Inspect Resume + AI Interview Scores -> Update Application Status -> Schedule Interview with Meeting Link -> View Platform Analytics.

SECTION 9: SCALABILITY, EDGE CASES & "TRICKY" VIVA QUESTIONS

Client tampering, scaling to 50k users, cheating prevention, and roadmap

Q16: Can a user modify their role from "candidate" to "admin" in localStorage to hack the system?

No, this is impossible. The backend never trusts client-side storage. The backend cryptographically validates the JWT signature using the secret key (process.env.JWT_SECRET). If a user alters the role in their browser, the cryptographic signature check fails immediately with HTTP 401/403 Invalid Signature.

Q17: How would you scale this backend to handle 50,000 concurrent candidates?

We would implement horizontal scaling and asynchronous queues:

- Stateless Horizontal Scaling: Deploy multiple Express backend containers behind an NGINX or AWS ALB load balancer.
- PgBouncer Connection Pooling: Manage thousands of concurrent PostgreSQL queries without exhausting connection limits.
- Redis + BullMQ Background Workers: Offload heavy AI evaluations and PDF parsing to background worker queues so HTTP requests return immediately.
- CDN Caching: Cache public job listings on Cloudflare CDN to reduce repetitive database queries.

Q18: What are the current limitations of your project, and what is your future roadmap?

Current limitations include browser-dependent speech recognition and text-based code evaluation. Future roadmap features include: (1) In-browser sandboxed code execution engine (Monaco Editor + Docker microvm), (2) WebRTC video proctoring and emotion analysis, and (3) Multi-round panel interview simulations.

SECTION 10: RAPID VIVA 1-LINE CHEAT SHEET

Quick-fire answers for instant recall during viva defense

Request Authentication: JWT sent in Authorization: Bearer <token>, verified via jwt.verify() in middleware.

File Upload Pipeline: multer.memoryStorage() buffers PDF in RAM -> pdf-parse extracts text -> buffer uploaded to Supabase Storage.

AI JSON Enforcement: Zero-shot prompt schema + regex cleaner strips ```json ticks before JSON.parse().

SQL Injection Guard: Supabase JavaScript client executes parameterized queries natively under the hood.

Password Security: Salted and hashed using bcryptjs with 10 rounds — plaintext is never stored.

In-App Notifications: Application state changes insert rows into notifications table, queried by React navbar bell.

Application Lifecycle: 6-stage state machine: applied -> reviewed -> shortlisted -> interview_scheduled -> accepted / rejected.

AI Model & Tech Stack: Google Gemini 1.5 Flash + React 18 (Vite, Tailwind) + Node/Express + Supabase PostgreSQL.